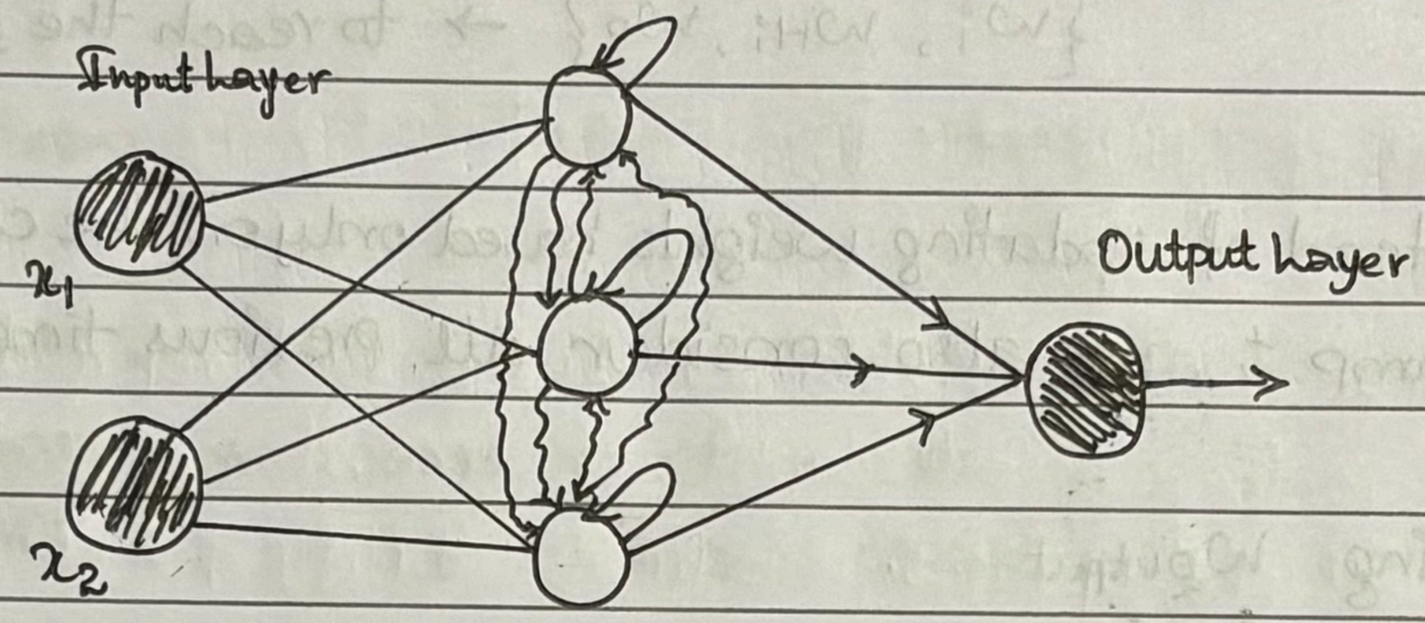
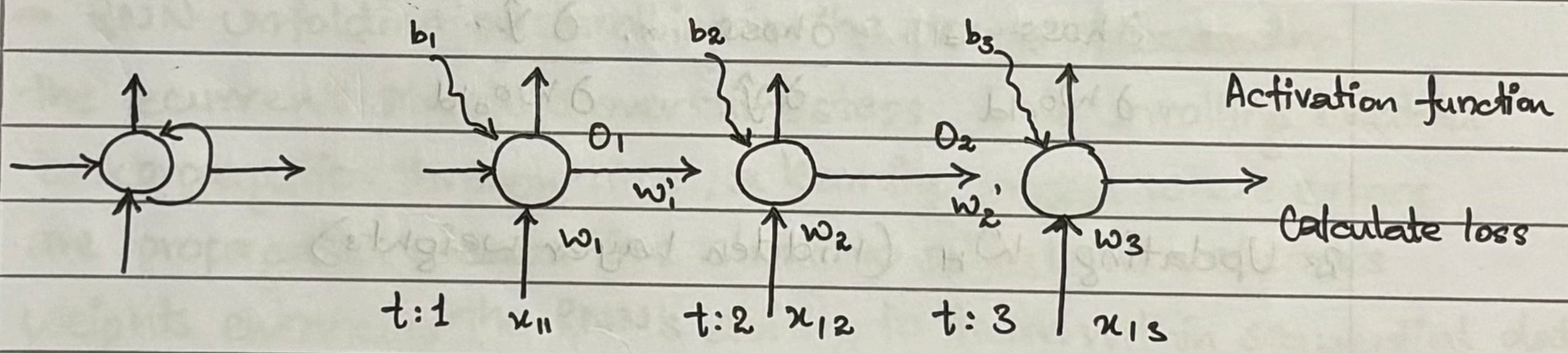
__/__/____

◦ RECURRENT NEURAL NETWORKS (RNN)

A RNN is a type of neural network designed to process sequential data, where the order of elements matters. Unlike traditional feed forward networks, RNNs have a memory that allows them to retain information from previous inputs, enabling them to understand context and dependencies within sequences.



① Forward Propagation with Time in RNN

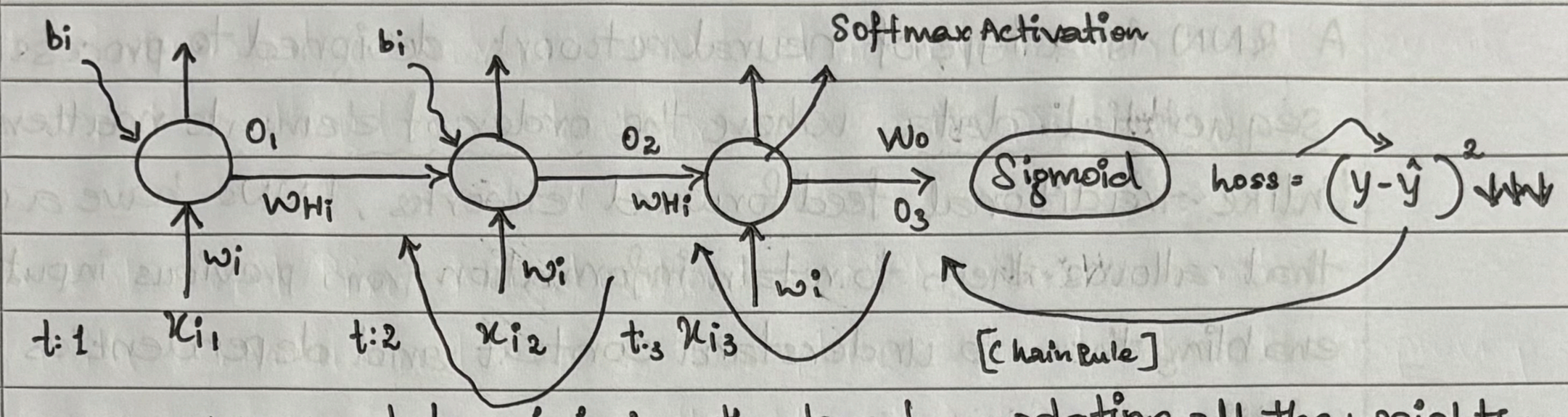


Forward Propagation: $O_1 = f(x_{11} \cdot w_1 + b_1)$ $\quad f = \text{ActFunc}()$

$O_2 = f(x_{12} \cdot w_2 + O_1 * w_1' + b_2)$

The fundamental processing unit in RNN is a Recurrent Unit. Recurrent units can remember information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.

② Backward Propagation with Time in RNN



we need to minimize the loss by updating all the weights.

$\{W_i, W_{Hi}, W_O\}$ → to reach the global minima

Here instead of updating weights based only on the current timestamp t, we also consider all previous timestamps: t-1, t-2...

1. Updating W_{output}

$$W_{O_{new}} = W_{O_{old}} - \eta \frac{\partial loss}{\partial W_{O_{old}}}$$

where,

$$\frac{\partial loss}{\partial W_{O_{old}}} = \frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial W_{O_{old}}}$$

2. Updating W_H (Hidden layer weights)

$$W_{H_{new}} = W_{H_{old}} - \eta \frac{\partial loss}{\partial W_{H_{old}}}$$

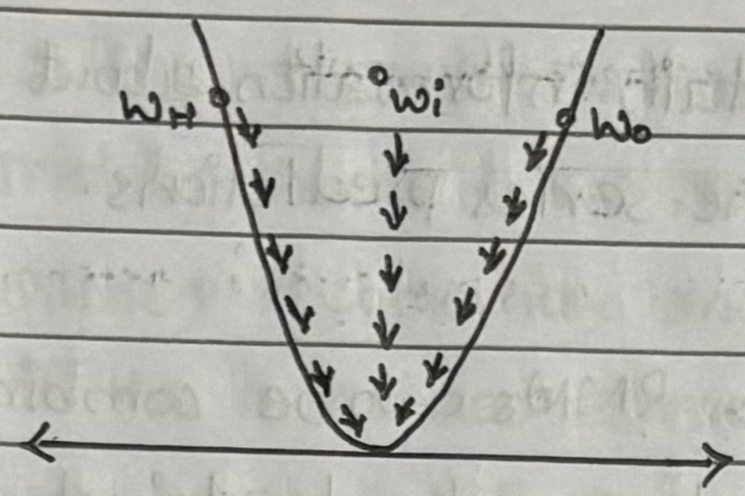
where,

$$\frac{\partial loss}{\partial W_{H_{old}}} = \frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3}^{t=3} \times \frac{\partial O_3}{\partial W_H} +$$

$$\frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial O_2}^{t=2} \times \frac{\partial O_2}{\partial W_H} + \frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \times \frac{\partial O_2}{\partial O_1} \times \frac{\partial O_1}{\partial W_H}$$

__/__/____

Our main aim is to reduce the loss function



3. Updating W_{input}

$$W_{inew} = W_{i_{old}} - \eta \frac{\partial loss}{\partial W_{i_{old}}}$$

where,

$$\frac{\partial loss}{\partial W_{i_{old}}} = \frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial W_{i_{old}}} + \frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \times \frac{\partial O_2}{\partial W_{i_{old}}} + \frac{\partial loss}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \times \frac{\partial O_2}{\partial O_1} \times \frac{\partial O_1}{\partial W_{i_{old}}}$$

→ RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. This unrolling enables back propagation through time, a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn within sequential data.

__/__/____

⇒ Advantages of RNNs

1. Sequential memory: RNNs retain information about previous inputs making them ideal for time-series predictions.

2. Enhanced Pixel Neighbourhoods: RNNs can be combined with convolutional layers to capture extendendpixels improving performance in image and video data processing.

⇒ Limitations of RNNs

1. Vanishing Gradient Problems: During backpropagation gradients dimish as they pass through each time stamp leading to minimal weight updates. This limits ability of RNN to learn long-term dependencies.

2. Exploding Gradient: Sometimes gradients grow uncontrollably causing excessively large weight updates that de-stabilize training.